

Training a Reinforcement Learning Agent with AutoRL for Traffic Signal Control

Yanjiao Xu

School of Computer Science and
Technology
Dalian University of Technology
Dalian, China
xuyanjiao@mail.dlut.edu.cn

Yuxin Wang *

School of Computer Science and
Technology
Dalian University of Technology
Dalian, China
wyx@dlut.edu.cn

Chanjuan Liu

School of Computer Science and
Technology
Dalian University of Technology
Dalian, China
chanjuanliu@dlut.edu.cn

Abstract — Traffic signal control system affects the efficiency of transportation. Adaptive traffic signal control has attracted attention for automatically adjusting the phase according to different traffic conditions. Researchers have been trying to apply deep reinforcement learning to the design of adaptive traffic signal control. The meta-parameter like γ , the discount rate of future reward, is crucial in reinforcement learning. Researchers, especially across disciplines, need to conduct extensive experiments to find appropriate values for the meta-parameters. Automated Reinforcement Learning (AutoRL) can automate the design choices of meta-parameters instead of manual tuning. The gradient-based meta-learning algorithm belongs to AutoRL. To save time in exploring meta-parameters, we integrate the gradient-based meta-learning algorithm into DQN (GBML-DQN). We conduct experiments on the traffic simulator SUMO. Our results show that GBML-DQN promotes convergence of the Q-value function and avoids overestimation to some extent, especially in the case of inappropriate training settings, whereas DQN fails in training.

Keywords — traffic signal control, reinforcement learning, AutoRL

I. INTRODUCTION

Traffic signal control system is responsible for coordinating vehicle flows running in different directions. Traditional traffic signal control systems are mostly based on a fixed phase scheme. In contrast, adaptive traffic signal control systems adjust the phase according to different traffic conditions automatically. With an increasing number of vehicles today, traffic conditions have become more complex and congestion can easily arise with traditional systems. Adaptive systems have been developed to improve transportation efficiency and have generally replaced traditional systems. Due to the reinforcement learning's ability of making intelligent decisions, some researchers have been trying to apply reinforcement learning to the design of adaptive traffic signal control systems [1,2].

In reinforcement learning, the design of state and reward [3], the choice of algorithms [4], and even the tricks of implementing an algorithm [5] result in different performance of a trained agent. Most studies focus on improving the performance of the agent and always neglect the training process. However, during the training process, researchers across disciplines tend to spend much more time on manual tuning of training-related parameters than experts in reinforcement learning who have prior experience in training a reinforcement learning agent.

Similar to hyperparameters in machine learning, meta-parameters in reinforcement learning are always fixed manually before training. [6,7] indicated that the training process in reinforcement learning is sensitive to meta-parameters like γ , the discount rate of future reward. Better than a fixed γ , starting at a low value and increasing γ repeatedly during training can help converge the algorithm [8]. Automated Reinforcement Learning (AutoRL) automates the design choices of meta-parameters instead of manual tuning [9]. It can help researchers free from redundant training processes to explore the appropriate meta-parameters, but focus on designing the algorithm itself.

The meta-gradient reinforcement learning is one of AutoRL methods. Due to its gradient-based characteristic, it is suitable for applying to DQN, which is a typical deep reinforcement learning widely used in traffic signal control studies. We integrate the gradient-based meta-learning algorithm into DQN (GBML-DQN) and conduct experiments on the traffic simulator SUMO.

II. BACKGROUND

In this section, we introduce the basic concepts of SUMO and DQN using the settings of our experiment as an example.

A. Traffic simulator - SUMO

SUMO [10], a traffic simulator, is used in our experiment to simulate a 2-way single intersection as represented in Figure 1 (a). The numbers from 0 to 15 are lane ids. Lanes labeled with numbers in yellow are called controlled lanes, meaning the lanes are controlled by traffic signals. The lines in the intersection are possible routes for vehicles. We label these routes with ids from 0 to 11 clockwise. Phase determines on each route whether passing through vehicles or not. In Table 1 we list the phases. Phase states are traffic signal controls for 12 routes with increasing route IDs. The letters r, y, G refer to traffic signals of red, yellow and green, respectively. By switching phases, the traffic signal is able to control the flow of traffic at the intersection. To test the traffic signal control system, we set two types of vehicle flows as shown in Figure 1 (b) and switch them during the simulation. The red arrow indicates heavy traffic and the green arrow indicates light traffic. After setting the road network and demand, the traffic intersection simulation is constructed.

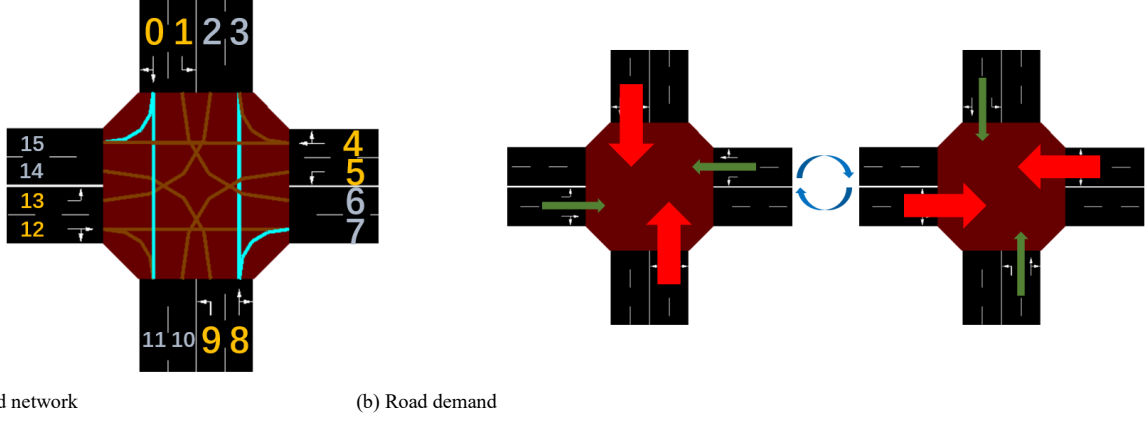


Figure 1. Road network and demand settings for the traffic simulation.

B. Deep reinforcement learning algorithm - DQN

In reinforcement learning [11], the agent learns by interacting with the environment. The agent takes action according to the state observed from the environment. The effect of the action on the environment returns a new state and reward to the agent. The reward guides the agent to adjust its action selection policy. After repeating this process, the agent is able to take better actions with the updated policy. Figure 2 depicts the above learning process and gives examples in gray to make the concept clear.

TABLE I. PHASE LIST.

Phase ID	State	Duration	Phase ID	State	Duration
0	GGrrrGGrrr	33	4	rrrGGrrrGGr	33
1	yyrrrryyrrr	2	5	rrryrrrryyr	2
2	rrGGrrrGrrr	33	6	rrrrGrrrrrG	33
3	rryyrrrrrrr	2	7	rrrrrrrrrry	2

In our experiment, the action refers to phase selection. There are 8 phases listed in Table 1, but only the phase IDs in 0, 2, 4, 6 are taken as actions in the implementation for simplicity. When switching the phase from state G to r, we program an additional phase state y as a transition period for safety reasons.

The state refers to a vector of controlled lane density. The density of a controlled lane d_i is defined in (1). n_i refers to the number of current vehicles on lane i . l_{lane_i} refers to the length of lane i and l_{veh} refers to the length of the vehicle.

$$d_i = \frac{n_i}{l_{lane_i}/l_{veh}} \quad (1)$$

Reward r is related to the number of waiting vehicles, defined in (2). w_i refers to the predicted number of waiting

vehicles after taking the action i and w refers to the actual number of waiting vehicles currently.

$$r = \begin{cases} 1, & \text{if } w = \min(w_0, w_1, w_2, w_3) \\ -1, & \text{otherwise} \end{cases} \quad (2)$$

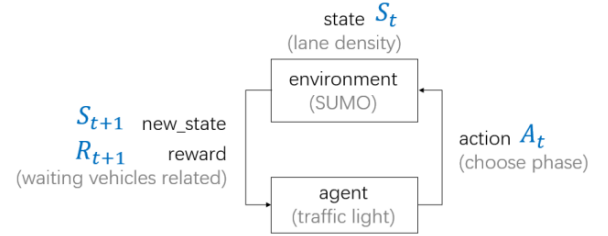


Figure 2. Basic concept of reinforcement learning.

The structure of DQN [12], a typical deep reinforcement learning algorithm, is depicted in Figure 3. To smooth the training distribution, DQN uses an experience replay mechanism. During the training process, DQN stores the transition (S_t, A_t, S_{t+1}, R_t) into the replay buffer and samples a batch transitions from the replay buffer for training. To improve the training efficiency, transitions that contribute more to the training are assigned higher sample probabilities, known as priority replay buffer. DQN uses the artificial neural network (ANN) to approximate Q-value function. The Q-value function is used for the agent to choose an action,

$$action = \operatorname{argmax}_a Q(S_t, a) \quad (3)$$

There are two neural networks in DQN. The policy network with parameters θ approximates the Q-value function. The target network with parameters θ_t duplicates the policy network periodically. The policy network updates for a more accurate Q-value function approximator by decreasing the Temporal Difference error (TD error),

$$TD \text{ error} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t) - Q(S_t, A_t; \theta) \quad (4)$$

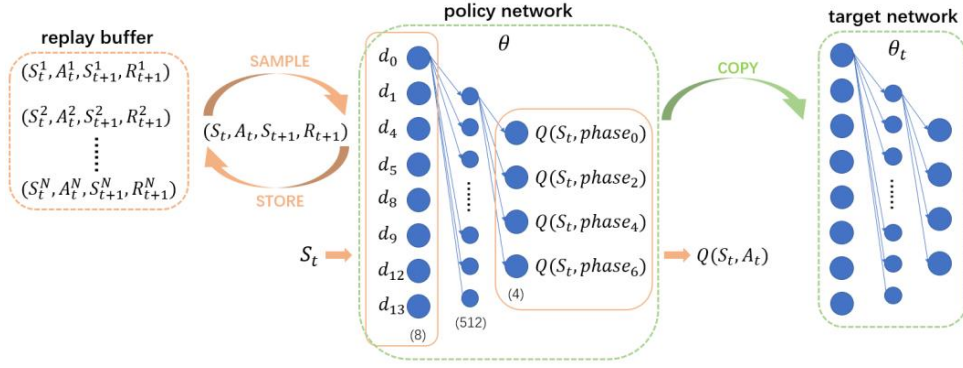


Figure 3. The structure of DQN.

III. GBML-DQN

We integrate the gradient-based meta-learning algorithm into DQN for adjusting γ automatically online. The core of the algorithm is calculating the gradient of γ with updated θ ,

$$\Delta\gamma = -\beta \frac{\partial J(\theta', \gamma_0)}{\partial \gamma} = -\beta \frac{\partial J(\theta', \gamma_0)}{\partial \theta'} \frac{d\theta'}{d\gamma} \quad (5)$$

θ' refers to the updated parameters of the policy network in DQN. β refers to the learning rate of γ . γ_0 refers to a fixed γ as a reference value. The change of θ is represented as a function $f(\theta, \gamma)$,

$$f(\theta, \gamma) = -\frac{\alpha}{2} \frac{\partial J(\theta, \gamma)}{\partial \theta} = \alpha (R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t) - Q(S_t, A_t; \theta)) \frac{\partial Q(S_t, A_t; \theta)}{\partial \theta} \quad (8)$$

In Equation (5), $d\theta'/d\gamma$ indicates how γ affects these updated parameters, and the gradient can be transformed to an accumulation form,

$$\frac{d\theta'}{d\gamma} = \frac{d(\theta + f(\theta, \gamma))}{d\gamma} = \frac{d\theta}{d\gamma} + \frac{\partial f(\theta, \gamma)}{\partial \gamma} + \frac{\partial f(\theta, \gamma)}{\partial \theta} \frac{d\theta}{d\gamma} = \left(I + \frac{\partial f(\theta, \gamma)}{\partial \theta} \right) \frac{d\theta}{d\gamma} + \frac{\partial f(\theta, \gamma)}{\partial \gamma} \quad (9)$$

To simplify Equation (9), we use $z = d\theta/d\gamma$, and Equation (9) is represented as,

$$z' = \mu z + \frac{\partial f(\theta, \gamma)}{\partial \gamma} \quad (10)$$

$\mu \in (0,1)$ refers to the decay of the accumulation trace to z . According to [13], choosing $\mu = 0$ results in a greedy meta-gradient that considers only the immediate effect of the meta-parameters γ on the parameters θ . $\mu = 0$ is often sufficient and can reduce the computation in practice. Then z' approximates to $\partial f(\theta, \gamma)/\partial \gamma$. Finally, Equation (5) is transformed to,

$$\Delta\gamma = -\beta \frac{\partial J(\theta', \gamma_0)}{\partial \theta'} z' \quad (11)$$

The process of updating the meta-parameter γ is presented in Algorithm 1. For more details, readers can view our open source code¹.

Algorithm 1. Integrating the gradient-based meta-learning algorithm into DQN.

Initialize the policy network with random parameters θ and duplicate it to the target network
Initialize replay buffer D to capacity N
Set the flag *update_gamma* to *False*
For *episode* = 1, M **do**
 For *simulation_step* = 1, T **do**

$$\theta' = \theta + f(\theta, \gamma) \quad (6)$$

In DQN, the loss function $J(\theta, \gamma)$ refers to the square of TD error,

$$J(\theta, \gamma) = (R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t) - Q(S_t, A_t; \theta))^2 \quad (7)$$

To simplify the representation of Equation (6), the gradient of the loss function $J(\theta, \gamma)$ to the parameters θ is defined as $f(\theta, \gamma)$; α refers to the learning rate of θ ,

In Equation (5), $d\theta'/d\gamma$ indicates how γ affects these updated parameters, and the gradient can be transformed to an accumulation form,

ϵ -greedy to choose an action
Take a simulation step with the action A_t
Add the state S_t , action A_t and observed new state S_{t+1} , reward R_{t+1} to D
If *update_gamma* == *False* **then**
 Randomly sample *batch_size* experiences from D
 Calculate z'
 Batch train the agent using (4); update θ to θ' ;
 periodically duplicate θ to θ_t
 Set *update_gamma* to *True*
Else
 Randomly sample another *batch_size* experiences from D
 Calculate $\frac{\partial J(\theta', \gamma_0)}{\partial \theta'}$
 Update $\gamma' = \gamma + \Delta\gamma = \gamma - \beta \frac{\partial J(\theta', \gamma_0)}{\partial \theta'} z'$
 Set *update_gamma* to *False*
End for
End for

IV. EXPERIMENTS AND RESULTS

During the experiments, we found that using the optimizer SGD with DQN was much better than RMSprop in training. By conducting experiments on different performances of the training process, it is clear to show the different effects of training with DQN and GBML-DQN.

¹ https://github.com/Desny/traffic_light_rl

We set all parameters for training to be the same, except for the optimizer. We define γ_0 as 0.95 and the learning rate α as 0.00025, β as 0.99. We train for 600 episodes on

SUMO v1.11.0, where each episode is 10000 seconds for traffic simulation. We define the batch size for training as 32 and the capacity of the replay buffer as 20000.

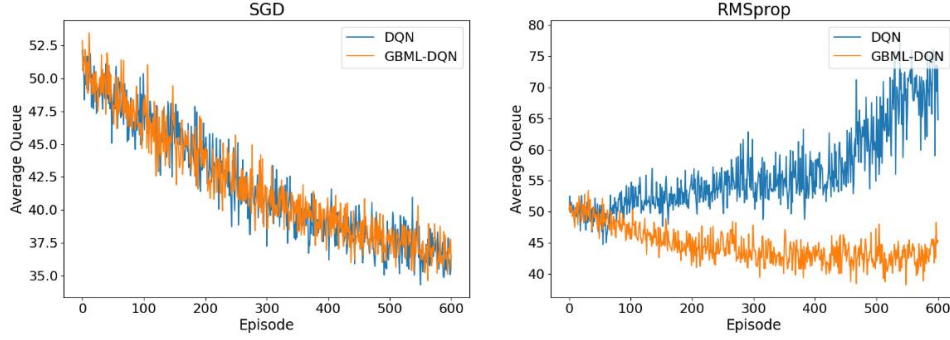


Figure 4. Compare the training performance between DQN and GBML-DQN.

In Figure 4, we compare the training performance of DQN and GBML-DQN by the average number of vehicles in the queue. With the optimizer SGD, the average queue continues to decrease from 52 to 35. The training performance of GBML-DQN is similar to that of DQN. However, there is a clear difference between DQN and GBML-DQN when the optimizer RMSprop is used. While the average queue of DQN increases, the average queue of GBML-DQN decreases. Even though it is not able to reduce the average queue to 35 as presented with SGD, GBML-DQN attempts to train the agent from an average queue of 50 to 40, while DQN fails when trained with RMSprop to an average queue above 70.

In Figure 5, to figure out the reason for the difference of these two algorithms in training performance, we relate the change of Q-value to γ 's. The top row in Figure 5 compares the change in Q-value and γ with the optimizer SGD between DQN and GBML-DQN. The change in the Q-value is similar. γ in GBML-DQN slightly changes between 0.9496 and 0.9501, in a small range of 0.0005. Compared to the fixed γ in DQN, Q-value has a tendency to converge around 0.5 with

an updating γ in GBML-DQN. The bottom row of Figure 5 compares the same with the optimizer GBML-DQN. Q-value fluctuates greatly with a fixed γ . In contrast, Q-value smoothly converges around 0.2 with γ changes in a wide range of 0.8.

V. CONCLUSION

We propose a GBML-DQN algorithm to train an agent for traffic signal control. The results show that GBML-DQN updates γ to promote the convergence of Q-value function and avoids the overestimation to some extent. It works particularly well with inappropriate training settings, while DQN fails during training. For researchers across disciplines, GBML-DQN can save them time in the training process by manually tuning the meta-parameters and focusing instead on the design of the algorithm.

In the future, we will test GBML-DQN on other single or group meta-parameters. To further improve the training process, we will pay attention to the development of AutoRL and integrate other AutoRL algorithms into reinforcement learning in traffic scenarios.

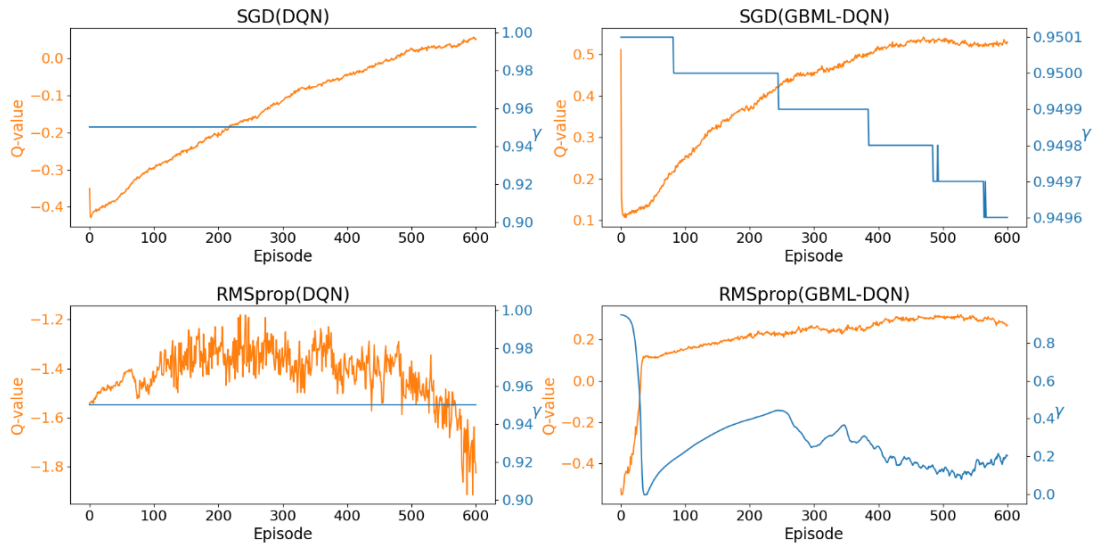


Figure 5. Compare the change in Q-value and γ .

REFERENCES

- [1] F. Rasheed, K. L. A. Yau, R. M. Noor, C. Wu, and Y. C. Low(2020). Deep Reinforcement Learning for Traffic Signal Control: A Review. *IEEE Access*.8:208016–208044.
- [2] K. L. A. Yau, *et al*(2017). A survey on Reinforcement learning models and algorithms for traffic signal control. *ACM Comput. Surv.*50(3).
- [3] W. Genders and S. Razavi(2016). Using a Deep Reinforcement Learning Agent for Traffic Signal Control.1–9, [Online]. Available: <http://arxiv.org/abs/1611.01142>.
- [4] Q. Zhao, C. Xu, and S. Jin(2019). Traffic Signal Timing via Deep Reinforcement Learning. *Smart Innov. Syst. Technol.*149(3):113-123.
- [5] D. H. Kim, J. H. Kim, and O. R. Jeong(2020). A study on the traffic signal control using the extended deep Q network. *ICIC Express Lett.*14(8):799-805.
- [6] P. Henderson, *et al*(2018). Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI)*.3207–3214.
- [7] M. Andrychowicz, *et al*(2021). What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations*.
- [8] D. V. Prokhorov and D. C. Wunsch(1997). Adaptive critic designs. *TNN*.8(5):997-1007.
- [9] J. Parker-Holder, *et al*(2022). Automated Reinforcement Learning (AutoRL): A Survey and Open Problems. *J. Artif. Intell. Res.*74:517-568.
- [10] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker(2012). Recent development and applications of SUMO - Simulation of Urban Mobility. *International Journal On Advances in Systems and Measurements*.5(3&4):128-138.
- [11] R. S. Sutton(2018). Reinforcement Learning: An Introduction, 2nd Edition.
- [12] V. Mnih, *et al*(2015). Human-level control through deep reinforcement learning. *Nature*.518(7540):529-533.
- [13] Z. Xu and D. Silver(2018). Meta-Gradient Reinforcement Learning.